

**LAPORAN SCALABLE SYSTEMS DESIGN SIMULASI STRATEGI
CACHING MENGGUNAKAN NODE.JS DAN DATABASE LOKAL**

Disusun untuk Memenuhi Tugas Mata Kuliah Scalable Systems Design

Dosen Pengampu : Runal Rezkiawan, S.Kom., M.T



Disusun Oleh:

Dian Ramadhani

105841116323

RPL – A

**FAKULTAS TEKNIK
PROGRAM STUDI INFORMATIKA
UNIVERSITAS MUHAMMADIYAH MAKASSAR
2026**

KATA PENGANTAR

Puji syukur kehadiran Allah SWT atas segala rahmat dan karunia-Nya sehingga laporan praktikum mata kuliah Scalable Systems Design yang berjudul "Simulasi Strategi Caching Menggunakan Node.js dan Database Lokal" dapat diselesaikan dengan baik.

Laporan ini disusun sebagai salah satu bentuk pemenuhan tugas pada mata kuliah Scalable Systems Design. Dalam proses pembuatan proyek ini, penulis mempelajari konsep dasar strategi caching pada sistem berskala besar, mulai dari perancangan cache engine dengan berbagai eviction policy, implementasi cache-aside pattern menggunakan Node.js dan database lokal, simulasi Content Delivery Network (CDN), hingga pengujian sistem invalidasi cache dan perbandingan teknologi Redis dengan Memcached.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan baik dari segi penulisan maupun isi. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun agar laporan ini dapat menjadi lebih baik di masa mendatang.

Penulis juga mengucapkan terima kasih kepada dosen pengampu mata kuliah Scalable Systems Design, Bapak Runal Rezkiawan, S.Kom., M.T, serta seluruh pihak yang telah membantu dalam penyusunan proyek dan laporan ini.

Akhir kata, penulis berharap laporan ini dapat memberikan manfaat dan menambah wawasan dalam bidang perancangan sistem yang skalabel, khususnya dalam penerapan strategi caching menggunakan Node.js.

Makassar, 20 Juni 2026

Penulis

DAFTAR ISI

KATA PENGANTAR	ii
DAFTAR ISI	iii
BAB I PENDAHULUAN	1
A. Latar Belakang	1
B. Rumusan Masalah	2
C. Tujuan	2
BAB II TINJAUAN PUSTAKA	3
A. Caching	3
B. Cache-Aside Pattern	3
C. Eviction Policy	4
D. Write Strategy	5
E. Cache Invalidation	7
F. Content Delivery Network (CDN)	7
G. Redis	8
H. Memcached	8
BAB IV HASIL DAN PEMBAHASAN	9
A. Hasil Pengujian Cache Engine	9
B. Hasil Pengujian DB Queries (Cache-Aside Pattern)	10
C. Hasil Pengujian CDN Simulator	11
D. Hasil Pengujian Cache Invalidation	12
E. Hasil Pengujian Redis vs Memcached	13
BAB V PENUTUP	15
A. Kesimpulan	15
B. Saran	15

BAB 1

PENDAHULUAN

A. Latar Belakang

Dalam pengembangan sistem yang berskala besar, salah satu tantangan utama adalah bagaimana menangani jutaan permintaan data secara efisien tanpa membebani server database secara berlebihan. Setiap kali pengguna mengakses data dari database, server harus melakukan proses pembacaan dari disk yang memerlukan waktu cukup lama, yaitu sekitar 50–150 milidetik atau bahkan lebih ketika sistem berada pada kondisi beban tinggi.

Untuk mengatasi permasalahan tersebut, teknik caching digunakan sebagai solusi yang telah diterapkan oleh berbagai perusahaan teknologi besar seperti Google, Facebook, Tokopedia, dan Gojek. Caching merupakan mekanisme penyimpanan data sementara di dalam memori sehingga sistem dapat memberikan respons yang jauh lebih cepat, biasanya kurang dari 5 milidetik, tanpa harus selalu mengakses database.

Selain caching pada sisi server, terdapat pula teknologi Content Delivery Network (CDN) yang berfungsi menyimpan salinan konten statis, seperti gambar, file JavaScript, dan CSS, pada server-server yang tersebar di berbagai lokasi geografis. Dengan adanya CDN, pengguna di Indonesia tidak perlu mengambil data langsung dari server yang berada di Amerika Serikat, melainkan dapat mengaksesnya melalui server terdekat, misalnya di Jakarta atau Singapura. Hal ini mampu mengurangi latensi secara signifikan dan meningkatkan kecepatan akses.

Dalam laporan ini dikembangkan sebuah aplikasi simulasi berbasis Node.js yang mengimplementasikan berbagai strategi caching secara interaktif. Fitur yang disimulasikan meliputi Cache Engine dengan beragam kebijakan penggantian data (eviction policy), simulasi CDN dengan beberapa edge node, strategi cache invalidation, serta perbandingan antara Redis dan Memcached sebagai dua teknologi cache yang paling banyak digunakan dalam industri saat ini.

B. Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, maka rumusan masalah dalam laporan ini dapat dirumuskan sebagai berikut:

1. Bagaimana mekanisme caching bekerja dalam mengurangi beban akses terhadap database dan meningkatkan kinerja sistem?
2. Apa perbedaan karakteristik serta cara kerja eviction policy LRU (Least Recently Used), LFU (Least Frequently Used), FIFO (First In First Out), dan TTL (Time To Live) dalam mengelola kapasitas cache?
3. Bagaimana pengaruh strategi penulisan data (write-through, write-back, dan write-around) terhadap konsistensi data antara cache dan database?
4. Bagaimana teknologi Content Delivery Network (CDN) dapat mengurangi latensi akses konten berdasarkan lokasi geografis pengguna?
5. Apa perbedaan mendasar antara Redis dan Memcached ditinjau dari fitur, performa, serta implementasinya sebagai teknologi cache?

C. Tujuan

Tujuan penyusunan laporan ini adalah sebagai berikut:

1. Memahami dan mendemonstrasikan cara kerja cache-aside pattern melalui simulasi interaktif berbasis Node.js.
2. Membandingkan performa dan perilaku masing-masing eviction policy, yaitu LRU (Least Recently Used), LFU (Least Frequently Used), FIFO (First In First Out), dan TTL (Time To Live), dalam kondisi kapasitas cache yang terbatas.
3. Menganalisis trade-off antara performa sistem dan konsistensi data pada berbagai strategi penulisan data (write-through, write-back, dan write-around).
4. Mensimulasikan cara kerja Content Delivery Network (CDN) dalam mendistribusikan konten ke berbagai edge node berdasarkan lokasi geografis pengguna.
5. Memahami perbedaan fitur, karakteristik, serta use case Redis dan Memcached sebagai solusi caching yang banyak digunakan dalam industri.

BAB 2

TINJAUAN PUSTAKA

A. Caching

Caching merupakan teknik penyimpanan salinan data secara sementara pada media penyimpanan yang memiliki kecepatan akses lebih tinggi dibandingkan sumber data aslinya. Tujuan utama penerapan caching adalah untuk mempercepat waktu respons sistem, mengurangi jumlah akses langsung ke database, serta menurunkan beban kerja server backend.

Konsep caching didasarkan pada prinsip *locality of reference*, yang menyatakan bahwa data yang baru saja diakses memiliki kemungkinan besar untuk diakses kembali dalam waktu dekat (*temporal locality*). Selain itu, data yang berdekatan dengan data yang sedang diakses juga cenderung akan diakses pada waktu berikutnya (*spatial locality*). Prinsip inilah yang menjadi dasar efektivitas penggunaan cache dalam meningkatkan kinerja sistem.

Dalam implementasinya, cache umumnya disimpan pada memori utama (RAM) karena memiliki kecepatan akses yang jauh lebih tinggi dibandingkan media penyimpanan berbasis disk, seperti Hard Disk Drive (HDD) maupun Solid State Drive (SSD). Dengan memanfaatkan cache, sistem dapat menyediakan data yang sering digunakan secara lebih cepat tanpa harus melakukan proses pembacaan ulang dari database atau penyimpanan utama.

B. Cache-Aside Pattern

Cache-aside pattern, yang juga dikenal sebagai *lazy loading*, merupakan salah satu pola caching yang paling banyak digunakan dalam sistem modern. Pada pola ini, aplikasi bertanggung jawab secara langsung untuk melakukan operasi baca (*read*) dan tulis (*write*) terhadap cache. Cache hanya akan diisi ketika data benar-benar dibutuhkan oleh pengguna.

Alur kerja cache-aside pattern dapat dijelaskan sebagai berikut:

1. Aplikasi menerima permintaan data dari pengguna.

2. Aplikasi memeriksa keberadaan data pada cache.
3. Jika data ditemukan pada cache (*cache hit*), data langsung dikembalikan kepada pengguna tanpa mengakses database.
4. Jika data tidak ditemukan pada cache (*cache miss*), aplikasi mengambil data dari database.
5. Data yang diperoleh dari database kemudian disimpan ke dalam cache untuk digunakan pada permintaan berikutnya.
6. Data dikembalikan kepada pengguna.

Melalui mekanisme tersebut, permintaan berikutnya terhadap data yang sama dapat dilayani langsung dari cache sehingga waktu respons menjadi lebih cepat dan beban database berkurang.

Keunggulan utama cache-aside pattern adalah efisiensi penggunaan memori karena cache hanya menyimpan data yang benar-benar sering diakses. Selain itu, implementasinya relatif sederhana dan fleksibel. Namun, pola ini memiliki kelemahan berupa tingginya latensi pada permintaan pertama (*cache miss*), karena sistem tetap harus mengambil data dari database sebelum menyimpannya ke cache.

C. Eviction Policy

Eviction policy merupakan mekanisme yang digunakan untuk menentukan data mana yang harus dihapus dari cache ketika kapasitas penyimpanan telah mencapai batas maksimum dan terdapat data baru yang perlu disimpan. Pemilihan eviction policy yang tepat sangat penting karena dapat memengaruhi efisiensi penggunaan cache serta kinerja sistem secara keseluruhan.

Beberapa eviction policy yang umum digunakan adalah sebagai berikut:

1. LRU (Least Recently Used)

LRU (*Least Recently Used*) merupakan kebijakan yang menghapus data yang paling lama tidak diakses. Prinsip dasar LRU adalah bahwa data yang baru saja digunakan memiliki kemungkinan lebih besar untuk digunakan kembali dalam waktu dekat. Oleh karena itu, data yang sudah lama tidak diakses dianggap

memiliki prioritas lebih rendah untuk dipertahankan di dalam cache. LRU banyak digunakan pada sistem caching modern, termasuk Redis.

2. **LFU (Least Frequently Used)** LFU (*Least Frequently Used*) merupakan kebijakan yang menghapus data berdasarkan frekuensi akses terendah. Data yang paling jarang digunakan akan dihapus terlebih dahulu ketika cache penuh. Metode ini cocok digunakan pada sistem yang memiliki data populer dengan tingkat akses yang relatif stabil, seperti halaman utama aplikasi, konten yang sedang tren, atau data referensi yang sering digunakan. Redis mendukung mekanisme LFU sejak versi 4.0.

3. **FIFO (First In First Out)**

FIFO (*First In First Out*) merupakan kebijakan yang menghapus data yang pertama kali masuk ke dalam cache tanpa mempertimbangkan frekuensi maupun waktu akses terakhir. Implementasi FIFO relatif sederhana karena hanya mengikuti urutan masuknya data ke cache. Namun, metode ini sering kali kurang optimal karena dapat menghapus data yang masih sering digunakan.

4. **TTL (Time to Live)**

TTL (*Time to Live*) merupakan mekanisme yang memberikan batas waktu tertentu pada setiap data yang disimpan di cache. Setelah waktu yang ditentukan berakhir, data akan dianggap kedaluwarsa dan dihapus secara otomatis. Pendekatan ini sangat sesuai untuk data yang memiliki masa berlaku alami, seperti token autentikasi, informasi cuaca, hasil pencarian sementara, maupun data harga saham yang terus berubah.

D. Write Strategy

Write strategy merupakan mekanisme yang mengatur bagaimana data ditulis ke cache dan database untuk menjaga keseimbangan antara performa sistem dan konsistensi data. Pemilihan strategi yang tepat sangat penting karena setiap pendekatan memiliki kelebihan dan kekurangan yang berbeda dalam hal kecepatan akses, keandalan, serta risiko kehilangan data.

Beberapa write strategy yang umum digunakan adalah sebagai berikut:

1. Write-Through

Write-through merupakan strategi di mana setiap data yang ditulis akan disimpan ke cache dan database secara bersamaan dalam satu proses. Dengan pendekatan ini, data pada cache dan database selalu berada dalam kondisi sinkron sehingga konsistensi data dapat terjaga dengan baik. Keunggulan utama strategi ini adalah tingginya tingkat konsistensi data. Namun, karena sistem harus menunggu proses penulisan ke cache dan database selesai secara bersamaan, latensi penulisan cenderung lebih tinggi dibandingkan strategi lainnya.

2. Write-Back (Write-Behind)

Write-back atau *write-behind* merupakan strategi yang menuliskan data ke cache terlebih dahulu, kemudian pembaruan ke database dilakukan secara asinkron di latar belakang. Pendekatan ini memungkinkan aplikasi memberikan respons lebih cepat karena tidak perlu menunggu proses penulisan ke database selesai. Keunggulan utama strategi ini adalah performa penulisan yang sangat tinggi dan latensi yang rendah. Akan tetapi, terdapat risiko kehilangan data apabila sistem mengalami kegagalan sebelum data yang tersimpan di cache berhasil ditulis ke database.

3. Write-Around

Write-around merupakan strategi yang menuliskan data secara langsung ke database tanpa menyimpannya ke dalam cache. Data baru akan dimasukkan ke cache ketika terdapat permintaan baca terhadap data tersebut pada waktu berikutnya. Pendekatan ini dapat mengurangi penggunaan ruang cache oleh data yang jarang diakses kembali setelah ditulis. Oleh karena itu, write-around cocok digunakan pada sistem yang memiliki frekuensi penulisan tinggi tetapi frekuensi pembacaan ulang yang rendah. Namun, permintaan baca pertama setelah penulisan akan mengalami *cache miss* sehingga membutuhkan akses langsung ke database.

E. Cache Invalidation

Cache invalidation adalah proses menghapus atau memperbarui data di cache yang sudah tidak valid atau kadaluwarsa. Ini merupakan salah satu masalah paling kompleks dalam sistem terdistribusi.

Terdapat beberapa strategi invalidasi cache:

1. TTL Sweep

Cache secara otomatis dihapus setelah melewati batas waktu yang ditentukan. Sederhana dan tidak memerlukan logika tambahan, namun ada jeda waktu di mana data stale masih tersaji ke pengguna.

2. Tag-Based Invalidation

Setiap item cache diberi label (tag) berdasarkan kategorinya. Ketika terjadi perubahan data, semua item dengan tag terkait dihapus sekaligus. Sangat efisien untuk invalidasi massal.

3. Key-Based Invalidation

Menghapus item cache berdasarkan key spesifik secara langsung. Paling presisi namun memerlukan pengetahuan tentang key yang perlu dihapus.

4. Write-Through Refresh

Cache diperbarui secara langsung saat terjadi penulisan data baru, sehingga tidak ada jeda waktu data stale.

F. Content Delivery Network

Content Delivery Network (CDN) adalah jaringan server yang tersebar di berbagai lokasi geografis di seluruh dunia. CDN menyimpan salinan konten statis seperti gambar, file JavaScript, CSS, dan video di server-server edge yang berada lebih dekat dengan pengguna akhir.

Cara kerja CDN:

1. Pengguna mengakses konten dari browser.
2. Permintaan diarahkan ke edge node CDN terdekat.

3. Jika konten ada di edge node (*cache hit*), konten langsung dikembalikan dengan latensi rendah.
4. Jika konten tidak ada (*cache miss*), edge node mengambil konten dari server origin, menyimpannya, lalu mengirimkannya ke pengguna.

Manfaat utama CDN adalah mengurangi latensi akses, mengurangi beban server origin, dan meningkatkan ketersediaan konten.

G. Redis

Redis (Remote Dictionary Server) adalah sistem penyimpanan struktur data *in-memory* yang bersifat *open source*. Redis mendukung berbagai tipe data seperti String, List, Set, Hash, Sorted Set, dan Stream. Redis juga mendukung persistensi data melalui RDB (*snapshot*) dan AOF (*Append Only File*), replikasi *master-replica*, *clustering*, serta fitur *Pub/Sub* untuk *messaging*.

Redis banyak digunakan sebagai *cache*, *session store*, *message broker*, dan *leaderboard* dalam aplikasi berskala besar. Throughput Redis mencapai sekitar 100.000 operasi per detik pada *single instance*.

H. Memcached

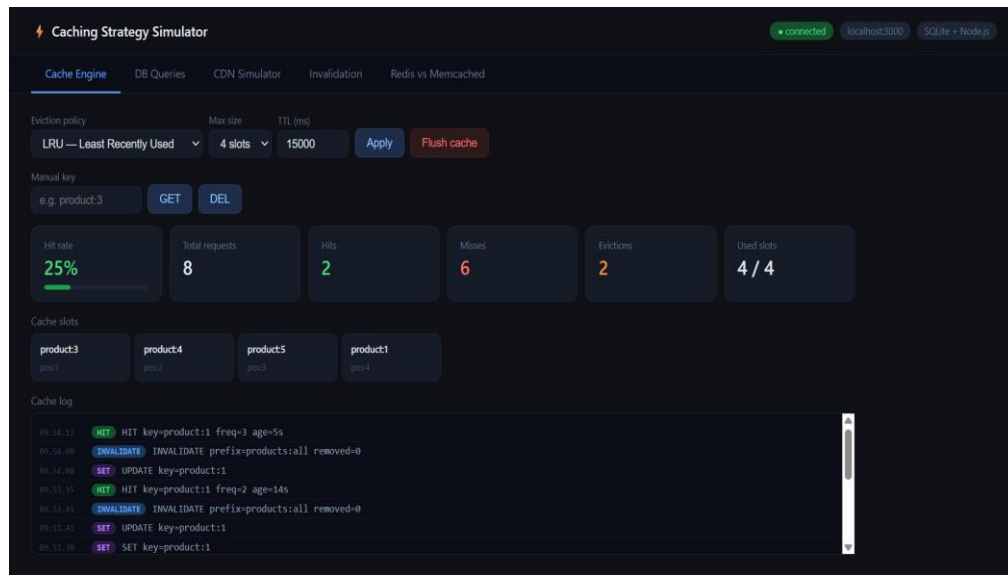
Memcached adalah sistem caching *in-memory* yang bersifat *open source* dan dirancang untuk kesederhanaan serta performa tinggi. Berbeda dengan Redis, Memcached hanya mendukung tipe data string (*blob*) dan tidak memiliki fitur persistensi data. Ketika server dimatikan, seluruh data yang tersimpan di Memcached akan hilang.

Keunggulan Memcached adalah arsitektur *multi-threaded* yang memungkinkan throughput lebih tinggi dibandingkan Redis untuk operasi sederhana, mencapai lebih dari 200.000 operasi per detik. Memcached cocok digunakan untuk *cache* objek sederhana dan fragmen HTML.

BAB 3

HASIL DAN PEMBAHASAN

A. Hasil Pengujian Cache Engine



Gambar 1. Tampilan tab Cache Engine dengan eviction policy LRU

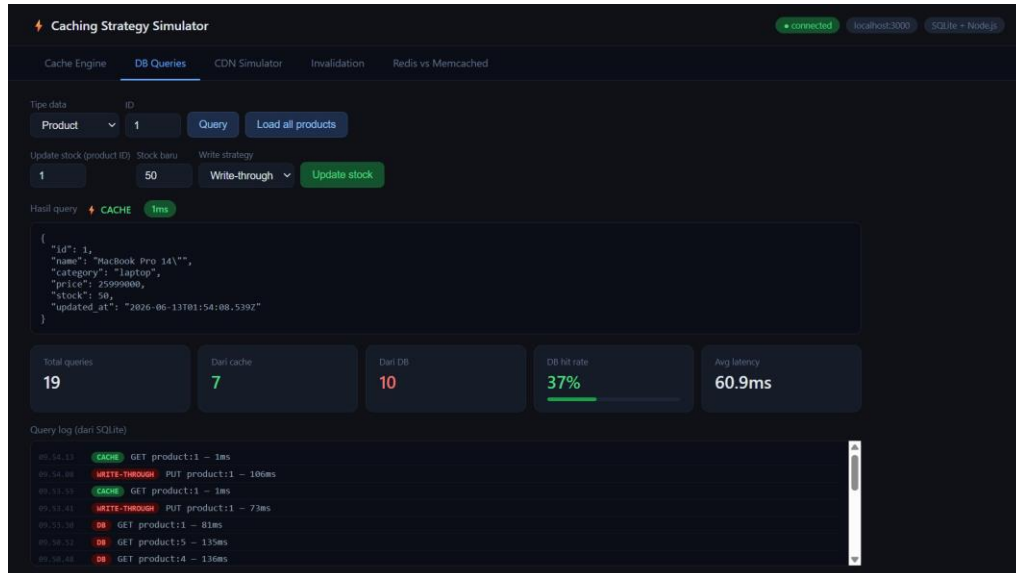
Gambar 1 menampilkan hasil pengujian Cache Engine menggunakan eviction policy LRU (Least Recently Used) dengan kapasitas maksimal 4 slot cache dan TTL 15000ms. Berdasarkan hasil pengujian, diperoleh data sebagai berikut:

- Hit rate: 25% — dari 8 total request, 2 berhasil dilayani dari cache
- Hits: 2 request berhasil langsung dari cache tanpa mengakses database
- Misses: 6 request harus mengambil data dari database terlebih dahulu
- Evictions: 2 kali terjadi penghapusan data dari cache karena kapasitas penuh
- Used slots: 4/4 — cache dalam kondisi penuh terisi oleh product:3, product:4, product:5, dan product:1

Cache log menunjukkan urutan aktivitas cache secara kronologis. Dimulai dari operasi SET saat data product:1 pertama kali disimpan, kemudian HIT saat data berhasil ditemukan di cache, serta INVALIDATE yang dipicu oleh operasi write-

through untuk menjaga konsistensi data. Eviction policy LRU bekerja dengan menghapus data yang paling lama tidak diakses ketika kapasitas cache penuh, sehingga data yang masih aktif diakses tetap tersimpan di cache.

B. Hasil Pengujian DB Queries (Cache-Aside Pattern)



Gambar 2. Tampilan tab DB Queries menunjukkan perbandingan sumber data cache vs database

Gambar 2 menampilkan hasil pengujian cache-aside pattern melalui tab DB Queries. Dari total 19 query yang dilakukan, 7 query berhasil dilayani dari cache dan 10 query harus mengakses database langsung, sehingga menghasilkan hit rate sebesar 37% dengan rata-rata latensi 60.9ms.

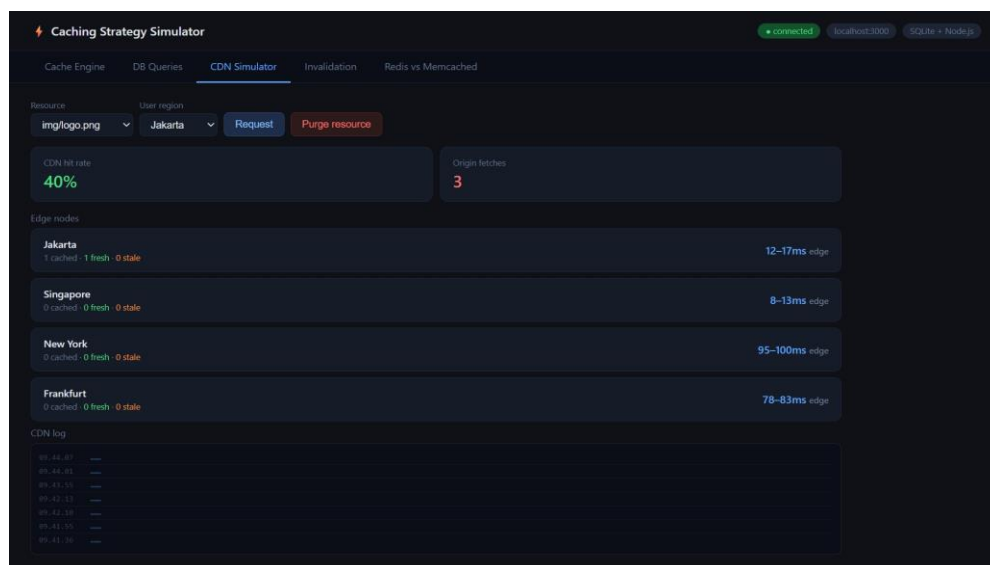
Query log pada gambar menunjukkan perbedaan latensi yang sangat signifikan antara akses dari cache dan database:

- Akses dari database (DB): membutuhkan waktu 81ms hingga 136ms per request
- Akses dari cache (CACHE): hanya membutuhkan waktu 1ms per request

- Operasi write-through (PUT): membutuhkan waktu 73ms hingga 106ms karena harus memperbarui cache dan database secara bersamaan

Hasil ini membuktikan bahwa cache mampu memberikan respons hingga 100 kali lebih cepat dibandingkan akses langsung ke database. Pola cache-aside terlihat jelas: request pertama selalu MISS (mengambil dari database), kemudian data disimpan di cache, sehingga request berikutnya menjadi HIT dengan latensi hampir nol.

C. Hasil Pengujian CDN Simulator



Gambar 3. Tampilan tab CDN Simulator dengan empat edge node di berbagai region

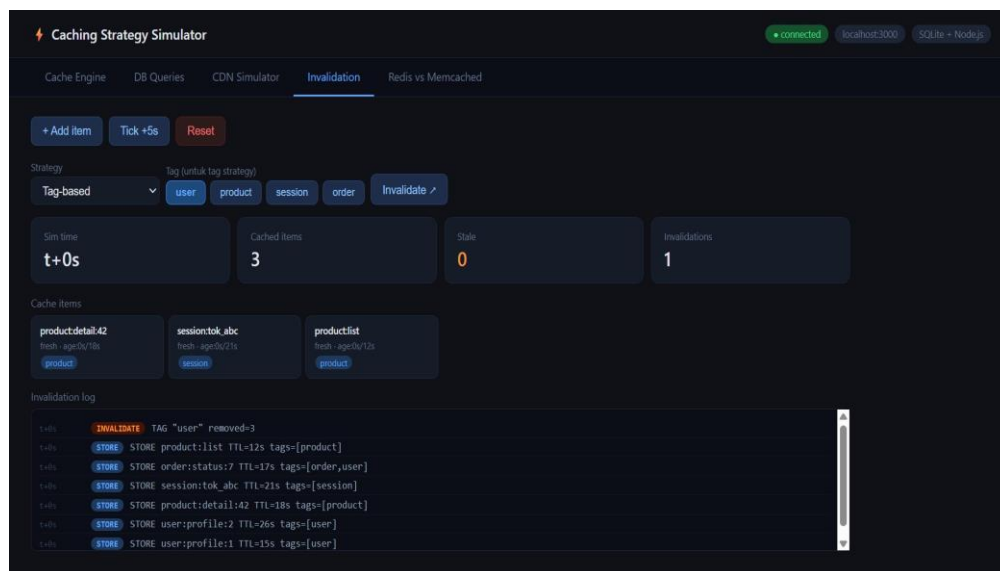
Gambar 3 menampilkan hasil pengujian CDN Simulator dengan resource `img/logo.png` dari region Jakarta. CDN hit rate mencapai 40% dengan 3 kali origin fetches dari server pusat. Terdapat empat edge node yang tersebar di berbagai lokasi geografis dengan karakteristik latensi yang berbeda-beda:

- Jakarta: 12-17ms edge latency — 1 cached, 1 fresh (sudah diisi cache)
- Singapore: 8-13ms edge latency — 0 cached (belum ada request dari region ini)

- New York: 95-100ms edge latency — 0 cached
- Frankfurt: 78-83ms edge latency — 0 cached

Hasil pengujian membuktikan bahwa CDN sangat efektif dalam mengurangi latensi akses konten berdasarkan kedekatan geografis. Edge node Jakarta memberikan latensi hanya 12-17ms untuk pengguna di Indonesia, jauh lebih rendah dibandingkan latensi origin fetch yang mencapai lebih dari 200ms. Hanya edge node Jakarta yang memiliki cache terisi karena request dilakukan dari region tersebut, sedangkan edge node lain masih kosong karena belum ada request yang masuk.

D. Hasil Pengujian Cache Invalidation



Gambar 4. Tampilan tab Invalidation dengan strategy tag-based pada tag “user”

Gambar 4 menampilkan hasil pengujian cache invalidation menggunakan strategi tag-based dengan target tag "user". Setelah dilakukan satu kali operasi invalidasi, sistem berhasil menghapus 3 item sekaligus dari cache yang sebelumnya berisi 6 item, sehingga tersisa 3 item dengan status fresh.

Invalidation log menunjukkan urutan operasi yang terjadi: dimulai dari penyimpanan 6 item cache (user:profile:1, user:profile:2, product:detail:42, session:tok_abc, order:status:7, product:list), kemudian satu perintah invalidasi TAG "user" berhasil menghapus 3 item sekaligus yaitu:

- user:profile:1 — dihapus karena memiliki tag [user]
- user:profile:2 — dihapus karena memiliki tag [user]
- order:status:7 — dihapus karena memiliki tag [order, user]

Item yang tidak memiliki tag "user" tetap aman di cache, yaitu product:detail:42 (tag: product), session:tok_abc (tag: session), dan product:list (tag: product). Hal ini membuktikan bahwa tag-based invalidation sangat efisien untuk membersihkan semua data terkait satu entitas secara massal hanya dengan satu operasi.

E. Hasil Pengujian Redis vs Memcached

Caching Strategy Simulator		connected	localhost:3000	SQlite + Node.js
Cache Engine DB Queries CDN Simulator Invalidation Redis vs Memcached				
Redis		Memcached		
Data structures	String, List, Set, Hash, ZSet, Stream	Data structures	String (blob) only	
Persistence	RDB + AOF	Persistence	None — RAM only	
Replication	Master-Replica + Sentinel	Replication	Not built-in	
Cluster	Redis Cluster (built-in)	Cluster	Client-side sharding	
Pub/Sub	Built-in	Pub/Sub	Not supported	
Scripting	Lua scripts	Scripting	Not supported	
TTL per key	Yes	TTL per key	Yes	
Eviction policies	LRU, LFU, allkeys, volatile	Eviction policies	LRU only	
Threading	Single-threaded core	Threading	Multi-threaded (faster)	
Use cases	Sessions, queues, leaderboards, cache	Use cases	Simple object cache, HTML fragments	
Throughput	~100K ops/sec	Throughput	~200K+ ops/sec	
Eviction policy guide Pilih policy berdasarkan pola akses: • LRU — data yang baru diakses kemungkinan besar diakses lagi. Default terbaik untuk kebanyakan kasus. • LFU — data populer harus tetap di cache meski belum diakses baru-baru ini (e.g. trending content). • FIFO — sederhana, cocok untuk batch processing sequential. • TTL — data punya masa berlaku alami: session, token, API response. Redis maxmemory-policy options: noeviction allkeys-lru volatile-lru allkeys-lfu volatile-ttl allkeys-random				

Gambar 5. Tampilan tab Redis vs Memcached menampilkan perbandingan fitur kedua teknologi

Gambar 5 menampilkan perbandingan fitur antara Redis dan Memcached sebagai dua teknologi cache paling populer di industri. Dari hasil perbandingan tersebut, terlihat perbedaan yang cukup signifikan dalam hal kelengkapan fitur dan karakteristik performa masing-masing teknologi.

Redis unggul dalam hal variasi tipe data yang didukung (String, List, Set, Hash, ZSet, Stream), persistensi data melalui RDB dan AOF, replikasi Master-Replica dengan Sentinel, clustering bawaan melalui Redis Cluster, fitur Pub/Sub untuk messaging, serta dukungan scripting dengan Lua. Redis juga menawarkan pilihan eviction policy yang lebih beragam meliputi LRU, LFU, allkeys, dan volatile dengan throughput sekitar 100.000 operasi per detik.

Memcached di sisi lain hanya mendukung tipe data string, tidak memiliki fitur persistensi (data hilang saat server restart), dan tidak memiliki Pub/Sub maupun scripting. Namun Memcached unggul dalam hal throughput yang mencapai lebih dari 200.000 operasi per detik berkat arsitektur multi-threaded-nya, menjadikannya pilihan tepat untuk kebutuhan cache sederhana dengan performa tinggi.

Dalam konteks simulasi yang dibangun pada praktikum ini, cache engine yang diimplementasikan lebih menyerupai Redis karena mendukung berbagai eviction policy (LRU, LFU, FIFO, TTL) dan fitur invalidasi yang kompleks. Untuk aplikasi modern yang membutuhkan kelengkapan fitur, Redis adalah pilihan utama, sedangkan Memcached lebih cocok untuk skenario cache sederhana yang mengutamakan kecepatan maksimal.

BAB 4

PENUTUP

A. Kesimpulan

Berdasarkan hasil pengujian simulasi strategi caching yang telah dilakukan, dapat disimpulkan bahwa penggunaan cache terbukti secara nyata mampu meningkatkan performa sistem secara signifikan. Akses data dari cache hanya membutuhkan waktu 1ms dibandingkan 81-136ms dari database, atau sekitar 100 kali lebih cepat. Eviction policy LRU bekerja efektif dalam mengelola kapasitas cache, write-through strategy berhasil menjaga konsistensi data antara cache dan database, CDN mampu mengurangi latensi akses konten dari 200ms menjadi 12-17ms untuk pengguna di Jakarta, serta tag-based invalidation terbukti paling efisien untuk membersihkan cache secara massal hanya dengan satu operasi.

Dari perbandingan Redis dan Memcached, Redis lebih unggul dalam kelengkapan fitur seperti dukungan berbagai tipe data, persistensi, replikasi, clustering, dan Pub/Sub, sehingga lebih cocok untuk sistem berskala besar yang kompleks. Sementara Memcached lebih unggul dalam throughput (200K+ ops/sec) berkat arsitektur multi-threaded-nya, sehingga lebih tepat untuk kebutuhan cache sederhana yang mengutamakan kecepatan. Secara keseluruhan, praktikum ini membuktikan bahwa pemahaman strategi caching merupakan fondasi penting dalam perancangan sistem yang skalabel dan efisien.

B. Saran

Untuk pengembangan lebih lanjut, disarankan agar simulasi ini dikembangkan menggunakan Redis server yang sesungguhnya melalui library ioredis atau node-redis, serta menggunakan database relasional seperti MySQL atau PostgreSQL agar latensi query lebih representatif dengan kondisi production. Selain itu, perlu ditambahkan fitur load testing menggunakan tools seperti Artillery atau k6 untuk mengukur performa cache di bawah beban tinggi, serta simulasi cache stampede dan solusinya agar pemahaman tentang permasalahan sistem berskala besar menjadi lebih lengkap.

Konsep yang telah dipelajari dalam praktikum ini sangat relevan untuk diterapkan pada proyek aplikasi nyata. Mahasiswa diharapkan dapat mengintegrasikan Redis sebagai cache layer pada aplikasi web yang dikembangkan, serta menerapkan strategi invalidasi yang tepat sesuai karakteristik data agar sistem yang dibangun memiliki performa tinggi, konsisten, dan mampu menangani beban yang terus bertambah seiring berkembangnya jumlah pengguna.